

1. Project Title

Agentic Financial Advisor Bot for Investment Strategies

2. Introduction and Background

Recent advancements in investment platforms have democratized access to markets but introduced a new problem: information overload without actionable clarity.

Investors today have access to news, analysis, stock prices, and macroeconomic data, yet few tools convert this information into clear, personalized investment advice.

Traditional financial advisory services remain expensive and inaccessible to many. Meanwhile, current robo-advisors like Wealthfront and Betterment offer automated investment management but lack transparency and real-time advisory interaction.

Thus, there is an opportunity to build an autonomous financial advisory agent that:

- Pulls and interprets real-time financial data,
- Provides conversational, transparent recommendations,
- Explains predictions in simple language,
- Clearly states limitations and uncertainty.
- Self-evaluates predictions over time and learns.

3. Project Template Choice

This project follows the "Financial Advisor Bot" template:

"An agent analyses financial data for active portfolio management"

In this case:

- Tools = Financial data APIs + Predictive models + Agent framework (e.g., AutoGPT-style loops).
- Task = Deliver dynamic, evolving, explainable investment advice in human-readable language.

4. Objectives

- Build a live agent that fetches live financial data (e.g., stock prices, crypto).
- Implement an analysis engine using simple prediction algorithms (e.g., linear regression, time series models, reinforcement learning).
- Perform self-updating predictions with fresh data.
- Use a small, efficient local LLM to generate advice in natural language summarizing key data, predictions, and disclaimers.

- Self-assess prediction accuracy over time and adjust its confidence or strategies.
- Provide a simple, intuitive user interaction interface (CLI initially, expandable to web).

5. Motivation and Justification

There is a clear demand among casual investors for:

- Actionable advice without needing to interpret complex charts,
- Explanations that are trustworthy, transparent, and simple,
- A low-barrier interface (command-line, messaging, or web).

Existing solutions have shortcomings:

- Robo-advisors make opaque decisions.
- Financial news aggregators overwhelm users with raw data.
- Trading bots focus on automation, not advice.
- Chat-based models (e.g., ChatGPT with finance plugins) are too general or inaccurate.

Unlike static bots, an agentic financial advisor:

- Takes initiative (e.g., "Markets are volatile today, here's a safer plan"),
- Adapts goals (e.g., "Recalculating based on unexpected Fed announcement"),
- Provides continuous advice, not just one-shot predictions,
- Explains uncertainties naturally, increasing trust and usability.

Existing tools (robo-advisors, static trading bots, even ChatGPT plugins) are passive.

This agent fills the gap between fully human advisors and static bots: a transparent, and explainable financial assistant.

Thus, it fills a gap for retail investors who want clarity but not full automation.

6. Technical Design

6.1 Data Sources

- Yahoo Finance API (stocks, ETFs)
- Polygon.io, AlphaVantage or Bloomberg API (optional backup)
- CCXT (crypto markets — optional)
- News sentiment APIs (optional, e.g., Finnhub)
- Scheduled updates every X minutes/hours depending on urgency.

6.2 Agent Core Loop

- Perception: Pull and preprocess live financial data.
- Reasoning: Apply models to predict outcomes.
- Action Selection: Choose advisory outputs based on predictions.
- Natural Language Reporting: Pass planning output to LLM to produce clear advice.
- Self-Evaluation: Log past predictions and compare to real outcomes over time.

6.3 Analysis Engine

- Simple Baseline: Moving averages, RSI, basic technical indicators.
- Intermediate: Time-series forecasting models (e.g., ARIMA, Prophet).
- Advanced:
 - Reinforcement Learning Agent: Trained on historical financial environments (e.g., Q-learning).
 - Reservoir Computing alternative for chaotic dynamics.
- Sentiment Signals: Optional lightweight news/headline sentiment models.

6.3 Natural Language Output

Instead of templates only, the agent uses a small fine-tuned LLM like:

- Phi-2 (2.7B, from Microsoft, lightweight)
- Mistral-7B-Instruct (slightly bigger, if resources allow)
- TinyLlama (1.1B fine-tuned for summarization and instruction following)

Deployment options:

- Run locally (using HuggingFace transformers + quantization)
- Host on a local server via Ollama,

Pipeline:

- Agent outputs a reasoning plan (structured JSON/text) →
- Small LLM translates into natural, human-readable report ("explain like I'm five" style).

6.4 Retrieval Augmented Generation

To improve the contextual awareness of the advisor bot, a Retrieval-Augmented Generation (RAG) pipeline will be integrated.

- Local Knowledge Base: Financial news articles, earnings reports, macroeconomic updates, indexed and refreshed daily.
- Retrieval: When making decisions, the agent will first retrieve relevant documents (using semantic search) to inform its advice generation.
- Augmentation: Retrieved documents will be passed along with the agent's reasoning output to the local LLM (e.g., Phi-2) to generate enriched, context-aware recommendations.
- Implementation:
 - OpenWebUI will be used to manage local LLMs and document retrieval pipelines.
 - Vector storage (e.g., ChromaDB, FAISS) for storing embeddings.
 - Embedding models such as MiniLM or E5-small-v2 can be used for document retrieval.

This allows the agent to generate more up-to-date, context-rich, and explainable financial advice.

6.5 Frontend

- Stage 1 (for POC): Command-Line Interface (CLI).
- Stage 2: Simple web front-end (Flask or FastAPI).
- Stage 3 (optional extension): Integration to apps like WhatsApp or Discord

6.6 Security and Disclaimer

- The bot will explicitly disclaim that predictions are purely informational, not financial advice.
- No trading actions will be performed — purely informational advisory.

7. Expected Output

An example output:

**"ALRIGHT, AFTER PULLING THE FRESHEST DATA:
CURRENT AAPL PRICE IS \$209.28 (APRIL 25TH CLOSE).
AFTER-HOURS IT'S SITTING AROUND \$208.60.
52-WEEK RANGE: \$169.11 TO \$260.10.
ANALYST 1-YEAR TARGET ESTIMATE: \$236.23.
🧙 MY PRICE PREDICTION FOR AAPL:
WITHIN THE NEXT 3 MONTHS, I PREDICT AAPL WILL RISE TO APPROXIMATELY**

\$226.45 BARRING MAJOR MARKET DISRUPTIONS.

DISCLAIMER: I'M NOT A FINANCIAL ADVISOR. THIS PREDICTION IS FOR INFORMATIONAL PURPOSES ONLY."

8. Evaluation Plan

Testing will focus on:

- Data Accuracy: Cross-check fetched data with primary sources.
- Model Performance: Test prediction accuracy on historical data.
- User Experience: Test clarity of outputs with sample users.
- Reliability: Ensure scheduled updates work without failure.

Unit tests and end-to-end testing will be performed on all modules (data, analysis, and output generation).

9. Related Work Review

Project	Strengths	Weaknesses/Gap Filled by Our Bot
Wealthfront	Automated portfolio management	No transparency; no daily advice
Robinhood Snacks	Digestible news summaries	No actionable investment advice
ChatGPT Stock Plugins	Conversational access to data	Lacks fine-tuned, reliable prediction logic
Crypto Bots (3Commas)	Auto trading with APIs	No conversational advice; crypto-only

By critically evaluating these, we see the missing space for simple, explainable investment advice bots, especially for casual investors.

10. Challenges and Risks

- Overfitting of predictive models on noisy financial data.
- Balancing prediction ambition with realism (avoid overpromising accuracy).
- Providing readable yet cautious advice (no financial liability).
- Dependency on external APIs' uptime and access limitations.

11. Timeline (approximate)

Week	Task
Week 1-2	Research and build data fetching system
Week 3-4	Implement simple prediction models
Week 5-6	Build templated natural language output
Week 7	Integrate full system, basic testing
Week 8	Refinement, user feedback, final testing
Week 9	Prepare final deliverables and documentation
